

Documentation - BTC Wallet Integration in DIAM Wallet

1. BTC Address Generation

- **Objective:** Generate a new Bitcoin address for the user.
- **Flow:**
 1. Utilize the **bitcoinjs-lib** library.
 2. Generate a new key pair.
 3. Derive the Bitcoin address from the public key.
 4. Return the generated address to the user.

2. BTC Balance Check

- **Objective:** Check the balance of a specific Bitcoin address.
- **Flow:**
 1. Accept the Bitcoin address as input.
 2. Use the BlockCypher API to fetch the balance of the provided address.
 3. Return the balance in BTC and its equivalent in USD to the user.

3. BTC Transfer/Transaction

- **Objective:** Transfer a specified amount of BTC from one address to another.
- **Flow:**
 1. Accept sender's private key, recipient's address, and amount in satoshis as input.
 2. Derive the sender's Bitcoin address from the private key.
 3. Fetch the UTXOs (Unspent Transaction Outputs) for the sender's address.
 4. Calculate the total available balance from UTXOs.
 5. Estimate the transaction fee.
 6. Check if the sender has enough balance to cover the transaction and fee.
 7. Create a new transaction, adding the recipient and amount.
 8. Calculate and add the change back to the sender's address.

9. Sign the transaction with the sender's private key.
10. Broadcast the transaction to the network.
11. Return the transaction hash (ID) to the user.

4. BTC Transaction History

- **Objective:** Fetch the transaction history of a specific Bitcoin address.
- **Flow:**
 1. Accept the Bitcoin address as input.
 2. Use the BlockCypher API to fetch the transaction history of the provided address.
 3. Return the list of transactions, including transaction IDs, amounts, and timestamps, to the user.

5. BTC Module Error Handling and Debugging

- **Objective:** Ensure smooth operation of the BTC module by handling potential errors and providing debugging information.
- **Flow:**
 1. Implement try-catch blocks in functions to handle potential errors.
 2. Provide descriptive error messages to inform the user of any issues.
 3. Log relevant information, such as fetched UTXOs, estimated fees, and derived addresses, for debugging purposes.
 4. Implement a rate-limiting mechanism to prevent rapid transaction attempts.